

Java Collections Framework - Complete Notes

Written in the same simple style we used in chat: why - how - when - interview.

These notes cover ArrayList, LinkedList, HashMap, HashSet, Queue, PriorityQueue and TreeMap.

1. Collections Framework - Big Picture

Java Collections Framework is a set of ready-made data structures. The idea is simple: use the right structure for the right job. Arrays are fixed in size, so Java introduced collections that can grow, keep order, keep uniqueness, or keep sorted order depending on the need.

Main families
List - ordered collection, duplicates allowed
Set - unique elements only
Queue - process in sequence
Map - key-value pairs

The most important interview habit is this: do not memorize a collection only by its definition. Always ask why it was created and what problem it solves.

2. ArrayList

ArrayList is a dynamic array. It was created because normal arrays have fixed size. If you create an array of size 10 and later need the 11th element, the array cannot grow. ArrayList solves this by resizing automatically when it becomes full.

Why does ArrayList exist?

Imagine cinema seats. Each seat has a number. If you want seat 50, you go directly to seat 50. That is why ArrayList is very fast for index-based access. It behaves like an array that can grow when needed.

ArrayList in one line
Dynamic array
Fast get(index)
Slow middle insert/remove because elements shift
Good when reads are more common than inserts in the middle

Internal working

Internally ArrayList stores data in an `Object[]` array. When space is available, the new element goes into the next free position. When the array becomes full, Java creates a bigger array, copies the old elements into it, and then continues. Resize is expensive, but it happens only occasionally.

ArrayList example:

```
List<Integer> list = new ArrayList<>();  
list.add(10);  
list.add(20);  
list.add(30);
```

Why is get(index) $O(1)$?

Because the data is stored in an array. Java can directly jump to the required index without checking earlier positions. That direct access is why ArrayList is excellent for reading by index.

Why is middle insertion slow?

Suppose the list is [10, 20, 30, 40]. If you insert 15 at index 1, Java must shift 20, 30 and 40 one step to the right. Because many elements may move, insertion in the middle is $O(n)$.

Time complexity

Operation	Complexity	Reason
get(index)	$O(1)$	Direct array access
add(end)	Amortized $O(1)$	Usually append, resize only sometimes
add(index)	$O(n)$	Elements shift
remove(index)	$O(n)$	Elements shift left
contains()	$O(n)$	Checks one by one

When to use ArrayList

Use ArrayList when you want frequent reads, fast access by index, and mostly append operations at the end.

Common mistakes

Many beginners think ArrayList is a linked structure. It is not. It is backed by an array. Another common mistake is thinking resize happens on every add. It only happens when the internal array becomes full.

3. LinkedList

LinkedList was created for frequent insertion and deletion. Unlike an array, elements are not stored continuously. Instead, each node points to the previous node and the next node.

Real-life analogy

Think of a train. Every coach knows the coach before it and the coach after it. That is exactly how LinkedList works.

Node structure:

```
prev <- [data] -> next
```

Internal working

LinkedList is a doubly linked list. Each node stores previous, data and next. Because of the two links, Java can move in both directions.

LinkedList in one line
Doubly linked list
Fast add/remove at head or tail
Slow random access
Good when the list changes often

Why is get(index) slow?

If the item is at index 500, Java must travel node by node to reach it. That traversal makes random access $O(n)$.

Why are insertions and removals fast?

Suppose you have 10 <-> 20 <-> 30. If 20 is removed, Java only changes the links so 10 points to 30. No shifting is needed. That is why insert/delete is fast when the position is already known.

Time complexity

Operation	Complexity	Reason
get(index)	$O(n)$	Traversal
addFirst/addLast	$O(1)$	Head/tail update
removeFirst/removeLast	$O(1)$	Head/tail update
add(index)	$O(n)$	Need to reach position first
contains()	$O(n)$	Sequential search

ArrayList vs LinkedList

ArrayList is better when you need fast random access. LinkedList is better when you insert and remove often, especially at the beginning or end. In interviews, always choose based on the most common operation.

4. HashMap

HashMap is one of the most important Java topics. Its goal is simple: find data quickly without searching every element. Instead of going item by item, HashMap jumps close to the correct place using hashing.

Why was HashMap created?

Suppose you have 10 lakh employees and you need employee 874512. If they are stored in a list, Java may need to check one by one. That is slow. HashMap avoids that by turning the key into a bucket index.

Internal structure

Internally HashMap uses an array called `table[]`. Every position in that array is called a bucket. A bucket may contain one node or many nodes if collisions happen.

Conceptual flow:

```
Key
-> hashCode()
-> bucket_index
-> bucket
-> node(s)
-> value
```

Bucket = array index

One of the biggest confusions is bucket. Bucket number is simply the array index. HashMap does not create a huge array using the raw hash code. It converts the hash into a valid index first.

HashMap put() flow

Key -> hashCode()

hash -> bucket index

If bucket is empty, add node

If bucket has nodes, use equals() to check the exact key

If same key exists, update value

put() and get()

put(): compute hash, go to bucket, compare keys, insert or overwrite. get(): compute hash, go to bucket, compare keys, return value if the exact key is found.

Collision

Collision happens when two different keys land in the same bucket. Java stores them in the same bucket. Before Java 8 this was a linked list. Java 8+ can convert a large bucket into a Red-Black Tree.

hashCode() vs equals()

hashCode() decides which bucket to go to. equals() decides which key inside that bucket is the exact match. That is why custom objects should usually override both methods.

Custom object idea:

```
class Student {
    int id;
    String name;

    // override equals() and hashCode()
}
```

Capacity, load factor and threshold

Default capacity is 16 and default load factor is 0.75. Threshold = capacity x load factor = 12. When size becomes 13, HashMap resizes to a bigger array.

Resize and rehash

Resize means creating a new larger array and moving all nodes to the new table. During rehashing, some nodes remain in the same bucket and some move to different buckets.

Treeification

In Java 8+, a bucket can become a Red-Black Tree when two conditions are met: bucket size is at least 8 and table capacity is at least 64. If the table is smaller than 64, Java prefers resizing first.

Time complexity

Operation	Average	Reason
put()	O(1)	Direct bucket lookup
get()	O(1)	Direct bucket lookup
remove()	O(1)	Direct bucket lookup
Worst case	O(n)	Heavy collisions
With treeification	O(log n)	Tree in bucket

Common mistakes

Do not think bucket means something separate from array index. Do not override only equals() or only hashCode(). Do not expect HashMap to keep order. Do not think resize simply copies data without rehashing.

5. HashSet

HashSet exists to store unique elements only. It does not have its own complex storage engine. Internally, it uses HashMap. The set element becomes the key and a dummy PRESENT object becomes the value.

Why was HashSet created?

If you keep checking duplicates in an ArrayList using contains(), the search is $O(n)$. HashSet gives fast duplicate checking using the hashing engine from HashMap.

Internal working

Conceptually:

```
set.add("Rahul")  
-> map.put("Rahul", PRESENT)
```

```
set.contains("Rahul")  
-> map.containsKey("Rahul")
```

How duplicates are prevented

When you add an element, HashSet uses hashCode() to find the bucket and equals() to check whether the exact same element already exists. If it already exists, the set does not add another copy.

HashSet in one line
Unique elements only
Built on HashMap
Element becomes key
PRESENT is dummy value

Custom objects in HashSet

If your custom object should be treated as the same based on some fields, override both equals() and hashCode(). Otherwise HashSet may treat logically equal objects as different.

Time complexity

Operation	Average	Reason
add()	$O(1)$	Hashing
contains()	$O(1)$	Direct bucket lookup
remove()	$O(1)$	Same as HashMap

HashSet vs ArrayList

ArrayList allows duplicates and supports index-based access. HashSet allows only unique elements and does not support indexing. Choose ArrayList when order and access by position matter. Choose HashSet when uniqueness matters.

6. Queue

Queue is a FIFO structure: First In, First Out. It models a real line, like a ticket counter or a support queue. The first element added is the first one removed.

Why does Queue exist?

Sometimes the order of processing matters. For example, a ticket queue or a printer queue should process in arrival order. Queue is designed for that exact pattern.

Queue operations

Method	Meaning
offer()	Add at the back
poll()	Remove from the front, return null if empty
peek()	View front without removing, return null if empty
remove()	Remove from front, throw exception if empty
element()	View front, throw exception if empty

Why LinkedList is commonly used for Queue

LinkedList can add at the tail and remove from the head in $O(1)$. That matches queue behavior nicely. The Queue interface hides random access because indexing does not fit the idea of a line.

Time complexity

Operation	Complexity	Reason
offer()	$O(1)$	Tail update
poll()	$O(1)$	Head update
peek()	$O(1)$	Read front
contains()	$O(n)$	Traversal

Queue vs Stack

Queue is FIFO. Stack is LIFO. Queue removes from the front. Stack removes from the top. If the question says first in first out, think Queue.

7. PriorityQueue

PriorityQueue is not FIFO. It removes elements based on priority. By default in Java, the smallest element has the highest priority. That is why PriorityQueue is often used when the order should depend on importance instead of arrival time.

Why was PriorityQueue created?

Imagine a hospital emergency room. A patient who arrives later may need treatment earlier because the condition is more serious. PriorityQueue handles this kind of problem better than a normal Queue.

Internal working

PriorityQueue uses a Binary Heap, usually a Min Heap by default. The smallest element stays at the root, so `peek()` can return it immediately. The elements are stored in an array, but the array represents a heap structure, not a fully sorted list.

`offer()`, `poll()`, `peek()`

`offer()` adds an element and then may restore the heap using heapify up. `poll()` removes the root, moves the last element to the root, then uses heapify down. `peek()` simply returns the root without removing it.

Time complexity

Operation	Complexity	Reason
<code>offer()</code>	$O(\log n)$	Heapify up
<code>poll()</code>	$O(\log n)$	Heapify down
<code>peek()</code>	$O(1)$	Root element
<code>contains()</code>	$O(n)$	Linear search

PriorityQueue vs Queue

Queue is FIFO. PriorityQueue is priority-based. Queue is often backed by LinkedList. PriorityQueue is backed by a heap. If you need the smallest or highest-priority item first, use PriorityQueue.

Custom priority

You can use a Comparator to change priority. For example, a reverse comparator makes the largest number come out first.

8. TreeMap

TreeMap stores key-value pairs like HashMap, but the keys are always kept sorted. It is useful when order matters and you want keys in ascending order by default.

Why was TreeMap created?

HashMap is fast, but it does not keep keys sorted. TreeMap is used when sorted order is more important than maximum lookup speed.

Internal working

TreeMap uses a Red-Black Tree, which is a self-balancing Binary Search Tree. This keeps search, insert and delete efficient. The tree stays balanced so operations remain $O(\log n)$.

put() and get()

put() compares the new key with existing keys and moves left or right depending on whether the key is smaller or larger. get() follows the same path until it finds the key.

Time complexity

Operation	Complexity	Reason
put()	$O(\log n)$	Tree traversal + balancing
get()	$O(\log n)$	Balanced tree search
remove()	$O(\log n)$	Balanced tree update

HashMap vs TreeMap

HashMap gives average $O(1)$ lookup and no sorting. TreeMap gives sorted keys and $O(\log n)$ operations. Use HashMap when speed matters most. Use TreeMap when ordered keys matter.

Comparator

If natural ordering is not enough, TreeMap can use a Comparator to define custom sorting logic.

9. Final Quick Revision

Collection	Main idea	Use it when
ArrayList	Dynamic array	Fast read by index
LinkedList	Doubly linked list	Frequent insert/remove
HashMap	Key-value with hashing	Fast lookup
HashSet	Unique elements	Duplicates should not be allowed
Queue	FIFO	Order matters
PriorityQueue	Priority based	Smallest/highest priority first
TreeMap	Sorted key-value	Keys must stay sorted

Interview rule: do not memorize only the definition. Always explain why the collection exists, how it works internally, and when to use it.

This is the real revision point before interviews.